

Projet INF-4393-01 été 2026 Structures des données et algorithmes

Exercice 1

Lorsqu'une action ordinaire d'une entreprise est vendue, le gain en capital (ou parfois la perte) correspond à la différence entre le prix de vente de l'action et le prix payé à l'origine pour l'acheter. Cette règle est facile à comprendre pour une seule action, mais si l'on vend plusieurs actions achetées sur une longue période, il faut alors déterminer précisément quelles actions sont vendues.

Un principe comptable standard pour identifier les actions vendues dans un tel cas consiste à utiliser le protocole **FIFO** (*First In, First Out* — premier entré, premier sorti) : les actions vendues sont celles qui sont détenues depuis le plus longtemps. D'ailleurs, il s'agit de la méthode par défaut utilisée dans plusieurs logiciels de finances personnelles.

Par exemple, supposons que nous achetions :

- 100 actions à 20 \$ chacune le jour 1 ;
- 20 actions à 24 \$ chacune le jour 2 ;
- 200 actions à 36 \$ chacune le jour 3 ;

Puis que nous vendions 150 actions à 30 \$ chacune le jour 4.

En appliquant le protocole FIFO, parmi les 150 actions vendues :

- 100 ont été achetées le jour 1 ;
- 20 ont été achetées le jour 2 ;
- 30 ont été achetées le jour 3.

Le gain en capital est alors :

[
100 * 10 + 20 * 6 + 30 * (-6)
]

Soit **940 \$**.

Travail demandé :

Écrivez un programme qui prend en entrée une séquence de transactions de la forme :

- « acheter x action(s) à y \$ chacune » (*buy x share(s) at \$y each*), ou
- « vendre x action(s) à y \$ chacune » (*sell x share(s) at \$y each*),

En supposant que les transactions ont lieu lors de jours consécutifs et que les valeurs x et y sont des entiers.

À partir de cette séquence de transactions, le programme doit produire en sortie le **gain (ou la perte) en capital total** pour l'ensemble des transactions, en utilisant le protocole **FIFO** pour identifier les actions vendues.

Exercice 2

Implémentez une classe **CardHand** qui permet à une personne d'organiser un ensemble de cartes dans sa main.

Le simulateur doit représenter la séquence de cartes à l'aide d'une unique **liste de positions (TDA Positional List)**, de manière à ce que les cartes de même couleur (*suit*) soient regroupées ensemble.

Implémentez cette stratégie au moyen de **quatre « doigts » (fingers)** pointant dans la main, un pour chacune des couleurs :

- cœurs (*hearts*),
- trèfles (*clubs*),
- piques (*spades*),
- carreaux (*diamonds*).

Ainsi, l'ajout d'une nouvelle carte dans la main ou le fait de jouer une carte d'une couleur donnée pourra être effectué en **temps constant $O(1)$** .

La classe doit fournir les méthodes suivantes :

- **addCard(r, s)** : ajoute une nouvelle carte de rang (*rank*) *r* et de couleur (*suit*) *s* à la main.
- **play(s)** : retire et retourne une carte de couleur *s* de la main du joueur. S'il n'existe aucune carte de cette couleur, retire et retourne une carte quelconque de la main.
- **iterator()** : retourne un itérateur permettant de parcourir toutes les cartes actuellement présentes dans la main.
- **suitIterator(s)** : retourne un itérateur permettant de parcourir toutes les cartes de couleur *s* actuellement présentes dans la main.

Remarque sur les « fingers »

Les *fingers* sont des références (positions/pointeurs) qui mémorisent un emplacement stratégique dans la liste pour chaque couleur. Par exemple :

[Cœur][Cœur][Cœur][Trèfle][Trèfle][Pique][Pique][Carreau]

Grâce à ces références, on peut accéder directement à la zone correspondant à une couleur donnée sans parcourir toute la liste, ce qui permet de réaliser les opérations addCard et play en **O(1)**.

Exercice 3

Écrivez un programme qui prend en entrée une **expression arithmétique entièrement parenthésée** (*fully parenthesized arithmetic expression*) et la convertit en un **arbre binaire d'expression** (*binary expression tree*).

Votre programme doit :

1. Construire l'arbre d'expression correspondant à l'expression donnée.
2. Afficher l'arbre d'une manière quelconque (par exemple sous forme hiérarchique ou traversée).
3. Calculer et afficher la valeur associée à la racine de l'arbre (c'est-à-dire le résultat de l'expression).

Défi supplémentaire

Permettez aux feuilles de l'arbre de contenir des variables de la forme : x1, x2, x3, ...

Ces variables sont initialement associées à la valeur 0.

Le programme doit ensuite permettre à l'utilisateur de modifier interactivement la valeur de ces variables. À chaque modification:

- la valeur de la variable concernée est mise à jour ;
- la valeur calculée à la racine de l'arbre d'expression est automatiquement recalculée et affichée.

Exemple

Entrée : $((3+5)*(2-1))$

Arbre :

```
      *
     /\
    + -
   /\ /\
  3 5 2 1
```

Valeur de la racine : $8 * 1 = 8$

Exemple avec variables :

Entrée : $((x1+5)*(x2-1))$

Valeurs initiales : $x1 = 0 ; x2 = 0$

Résultat : $(0 + 5) * (0 - 1) = -5$

Après mise à jour : $x1 = 3 ; x2 = 4$

Nouveau résultat : $(3 + 5) * (4 - 1) = 24$